Instructions

- Follow the learner instructions in the grey text boxes for each slide.
- Delete the text box and instructions when complete.

Instructions for learner:

- Action 1
- Action 2
- Action 3

Build a Personalized Online Course Recommender System with Machine Learning

Florian H.
22/04/2026

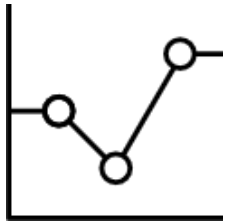
Outline

- Introduction and Background
- Exploratory Data Analysis
- Content-based Recommender System using Unsupervised Learning
- Collaborative-filtering based Recommender System using Supervised learning
- Conclusion
- Appendix

Introduction

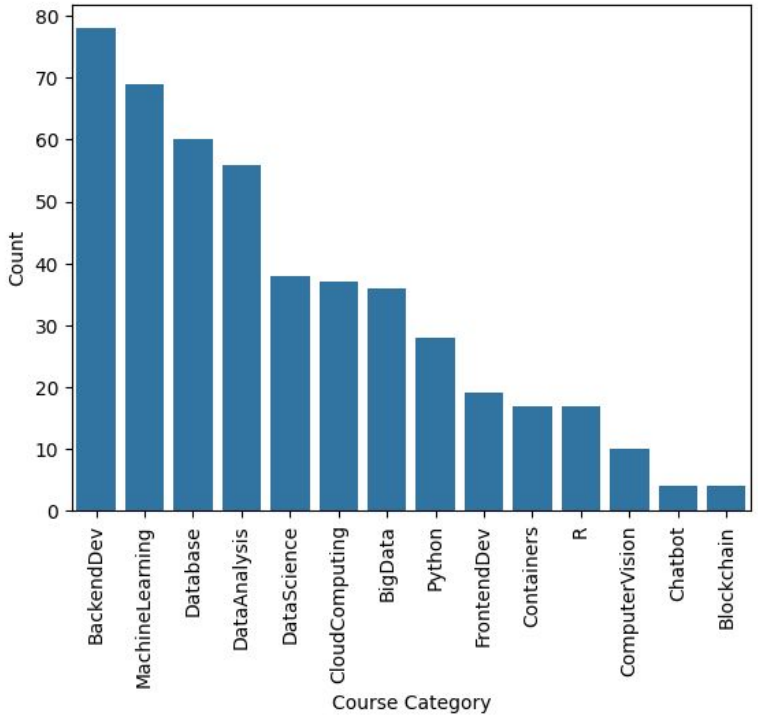
- Recommender systems are everywhere
- So it is important to understand how they work and how to evaluate their performance
- This presentation shows a comparison of multiple recommender architectures.
- An overview is provided of how they perform and what types of recommender systems are available.

Exploratory Data Analysis



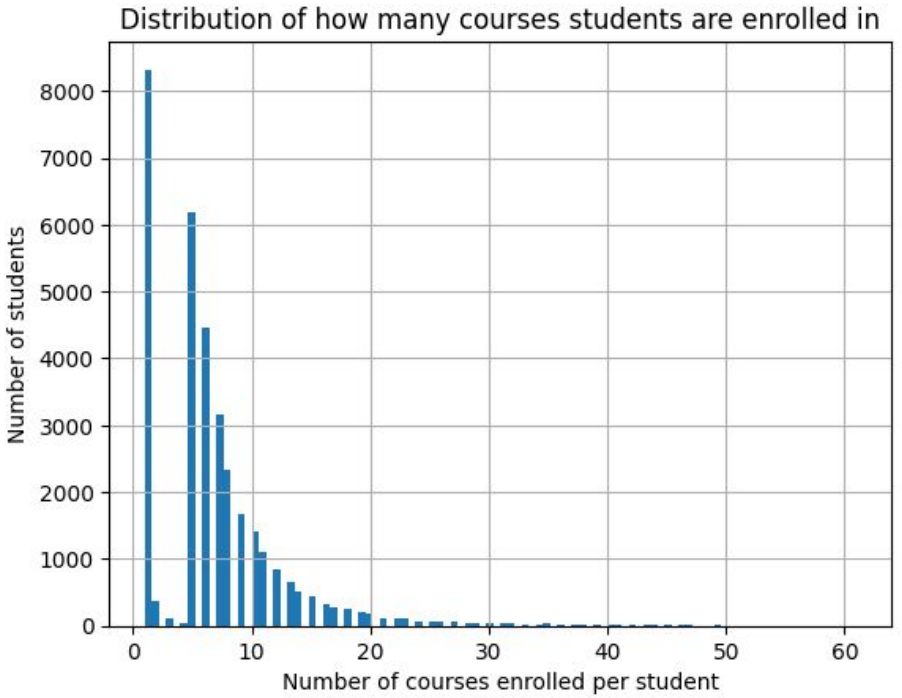
Course counts per genre

- The distribution of course genres shows that there is an imbalance between category counts.
- Most courses are in the categories of “Backend Development”, “Machine Learning”, “Database” and “Data Analysis”
- “Chatbots” and “Blockchain” are not represented by many courses
- This may cause issues later when recommending these courses.
- We should keep this in mind when performing further EDA



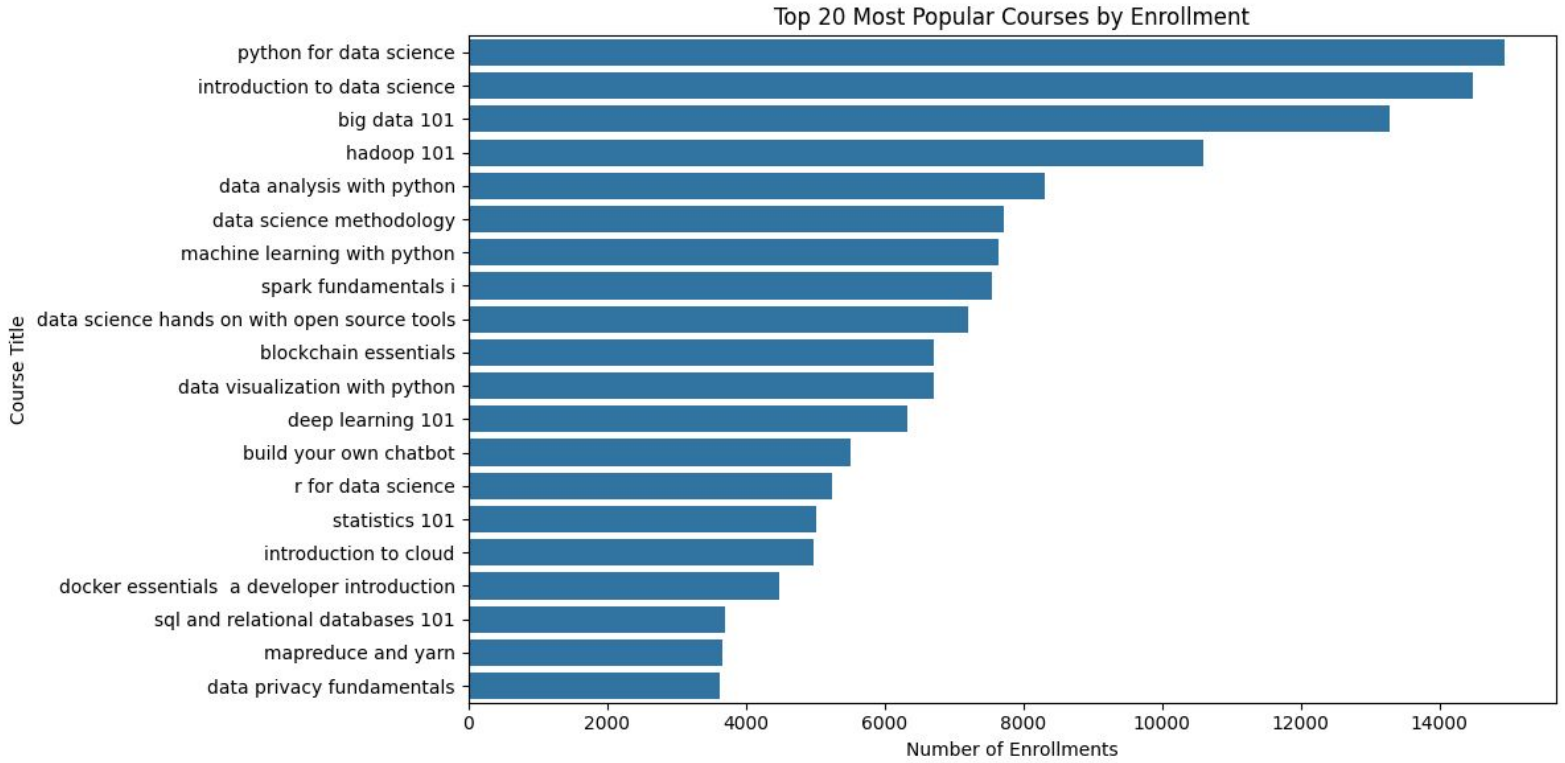
Course enrollment distribution

- Most students have enrolled in 5+ courses.
- A significant portion only enrolled in a single course.
- This again shows an imbalance which could cause problems for the recommender system.



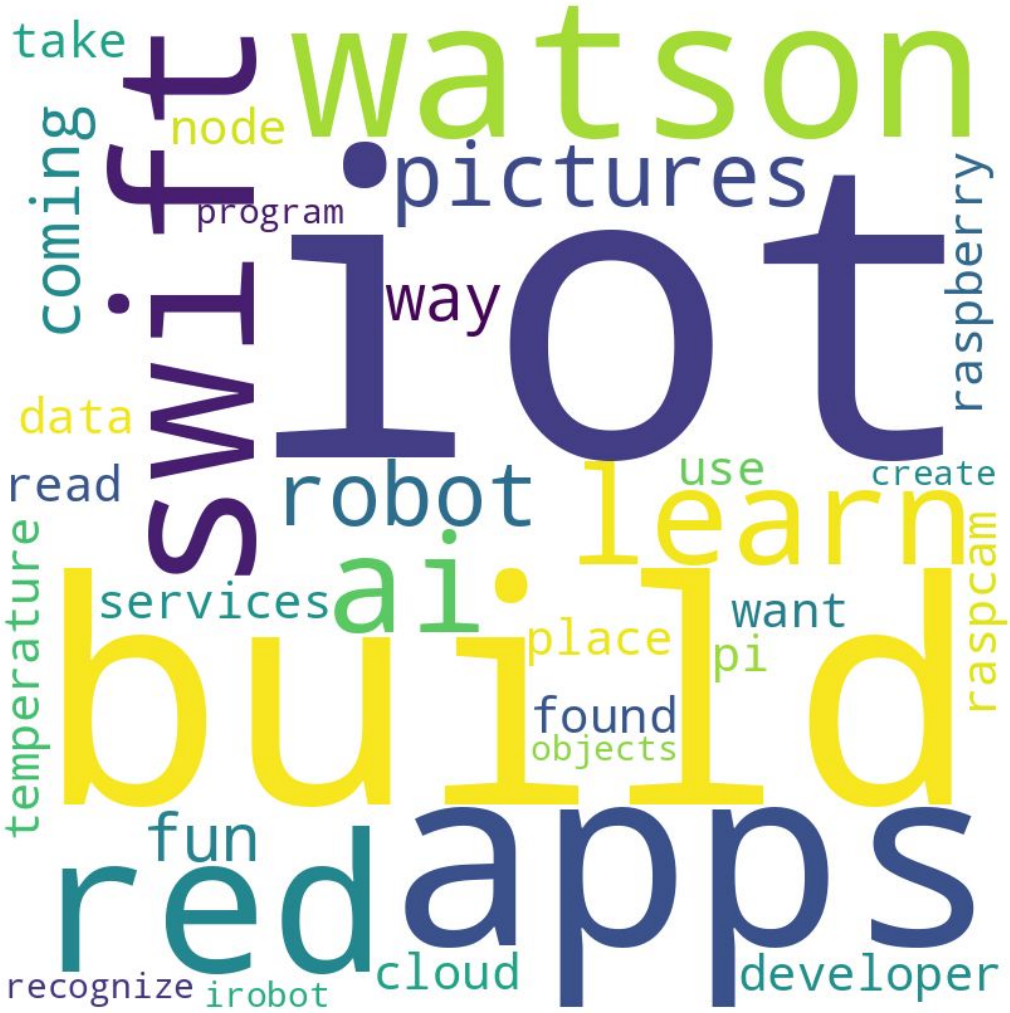
20 most popular courses

- The 20 most popular courses are reasonably well distributed.
- Some courses are very popular, mainly related to data science
- When building recommendations, this may influence uneven weighting between groups of recommendations
- For example, when clustering the recommendations may be skewed towards data science.

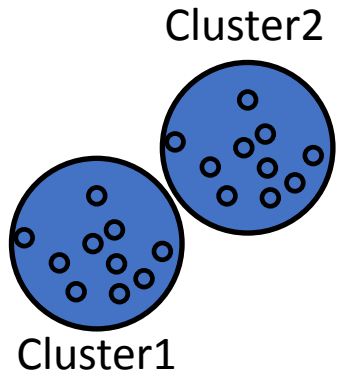


Word cloud of course titles

- Many courses are about IOT, Apps and building
- Other common topics seem to be AI, Watson and robotics



Content-based Recommender System using Unsupervised Learning

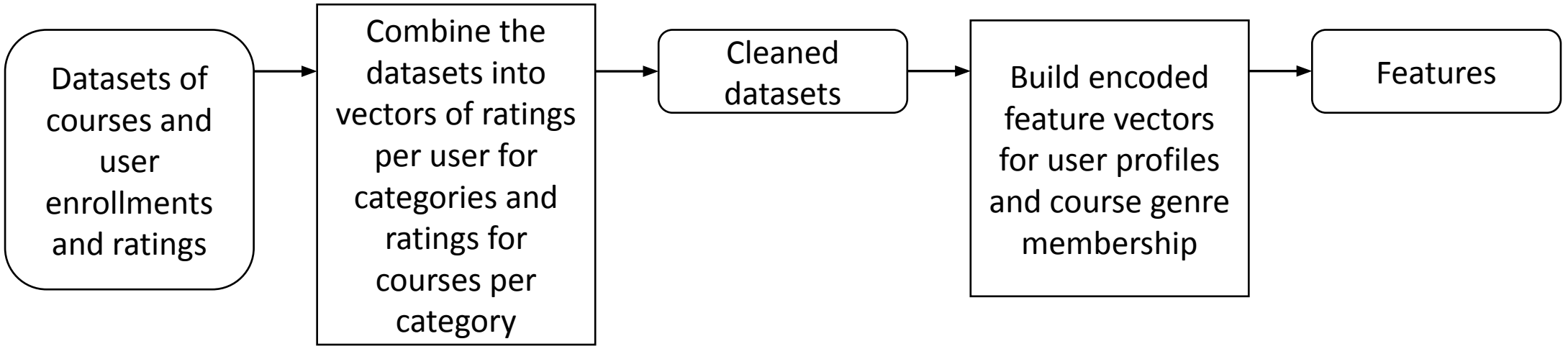


Flowchart of content-based recommender system using user profile and course genres

- First the data needs to be processed to a format using vectors of users and courses.
- Input data shape will become [users, categories] and [course, categories]

```
# Group the test users DataFrame by the 'user' column and find the maximum value for each group,
# then reset the index and drop the old index to obtain a DataFrame with unique user IDs
test_users = test_users_df.groupby(['user']).max().reset_index(drop=False)

# Extract the 'user' column from the test_users DataFrame and convert it to a list of user IDs
test_user_ids = test_users['user'].to_list()
```



Evaluation results of user profile-based recommender system

By trying a few threshold values, the value of 30 was chosen, this gave an average of ~20 courses recommended per user.

With this threshold value there are 20 courses recommended per user on average.

```
THRESHOLD = 30
res_df[res_df['SCORE'] > THRESHOLD].groupby(by="USER").size().mean()
```

0.0s

```
19.854561480828558
```

Most recommended courses:

'analyzing big data in r using apache spark',

'getting started with the data apache spark makers build',

'spark fundamentals ii',

'spark overview for scala analytics',

'using the cql shell to execute keyspace operations in cassandra',

'nosql systems',

'distributed computing with spark sql',

'cloud computing applications part 2 big data and applications in the cloud',

'foundations for big data analysis with sql',

'analyzing big data with sql'

Flowchart of content-based recommender system using course similarity

- The initial similarity matrix was provided

Evaluation results of course similarity based recommender system

Using the default value of 0.6 as a threshold for similarity

There are ~2 unseen courses recommended per user at this threshold.

```
THRESHOLD = 0.6
res_df[res_df['SCORE'] > THRESHOLD].groupby(by="USER").size().mean()
```

✓ 0.0s

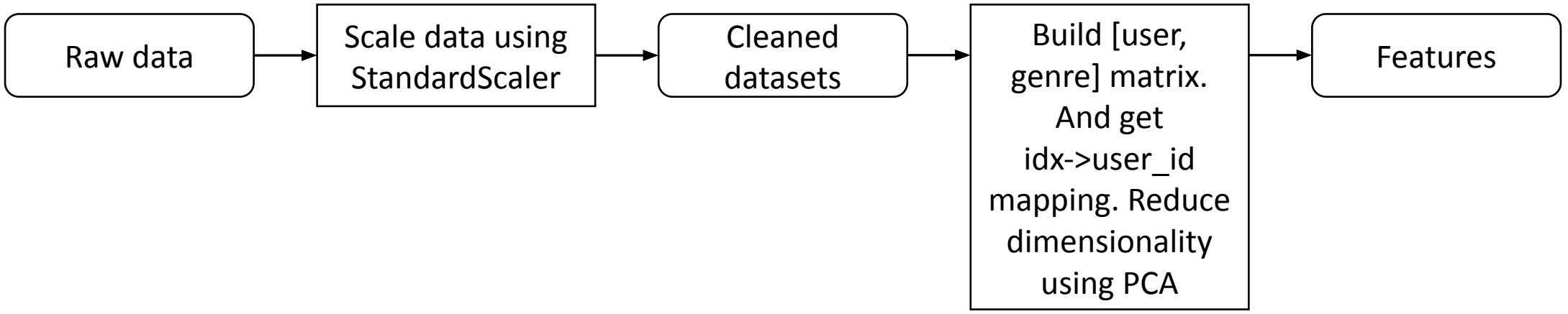
```
1.9528524028684577
```

Most recommended courses:

```
'watson analytics for social media',
 'text analytics 101',
 'deep learning with tensorflow',
 'data science with open data',
 'build your own chatbots',
 'deep learning with tensorflow',
 'introduction to data science in python',
 'introduction to data science in python',
 'a crash course in data science',
 'data science fundamentals for data analysts'
```

Flowchart of clustering-based recommender system

- Since we are now clustering, we need to ensure that the data is scaled to avoid biased clusters
- Then we again build the [User, Genre] matrix representing profile vectors.
- Later, an additional PCA preprocessing step was added to reduce data dimensionality



Evaluation results of clustering-based recommender system

For the course threshold, an enrollment number of 100 was chosen as the minimum for recommending a course.

On average, 30 courses were recommended.

```
# Average number of courses recommended
np.mean([len(recs) for recs in results.values()])
```

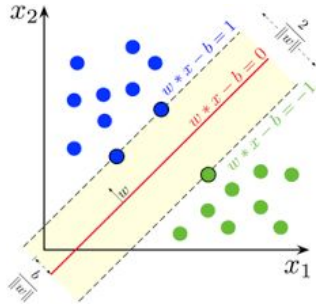
✓ 0.0s

```
30.601752160703224
```

The top 10 recommended courses were

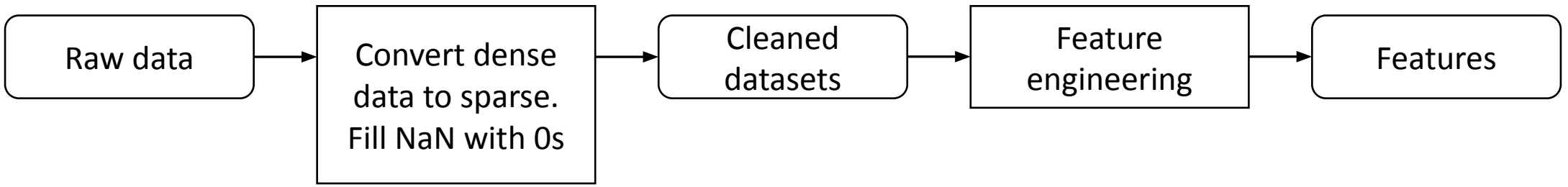
```
'watson analytics 101',
 'predictive modeling fundamentals i',
 'statistics 101',
 'ibm cloud essentials',
 'deep learning 101',
 'docker essentials  a developer introduction',
 'data privacy fundamentals',
 'introduction to cloud',
 'sql and relational databases 101',
 'r for data science'
```

Collaborative-filtering Recommender System using Supervised Learning



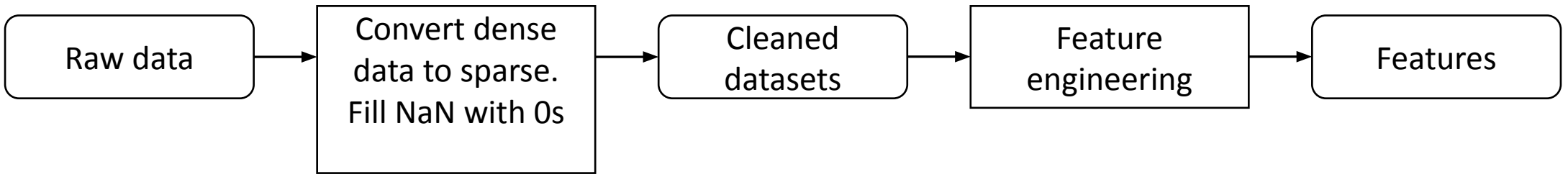
Flowchart of KNN based recommender system

- Sparse data is better for computations than dense data. This is because the sparse representation is a vector, which we can use with linear algebra.
- No feature engineering was necessary in this case since the data was already in the correct format with the required features.
- Then the surprise library was used to fit a KNN recommender system.



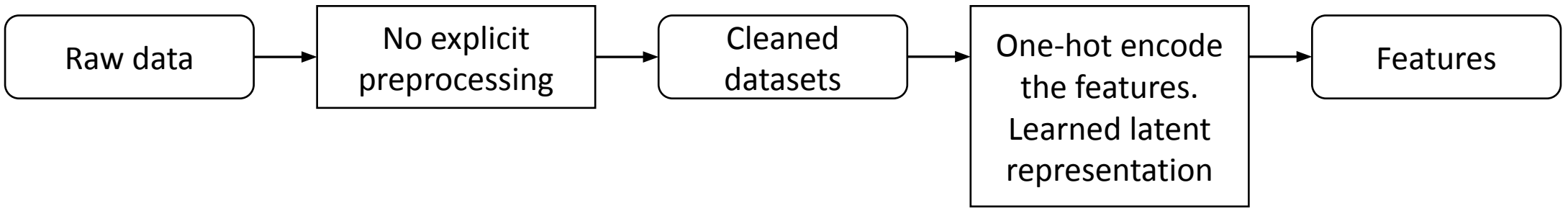
Flowchart of NMF based recommender system

- The same preprocessing step was used as in the KNN based recommender system.
- Again, no explicit feature engineering was needed.
- NMF reduces the dimensionality by fixing an intermediate representation. This representation is learned by way of two non-negative matrices.



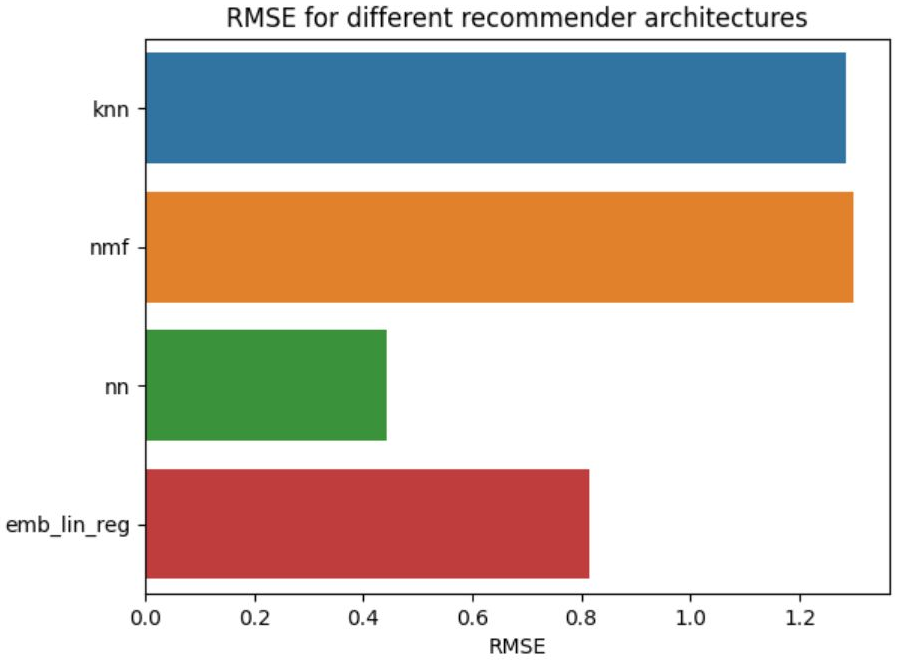
Flowchart of Neural Network Embedding based recommender system

- Since we are using a neural network, we need to encode the categorical features in a one-hot format.
- The network makes use of an embedding layer that encodes the input into a lower dimensional latent representation. This latent representation is learned at the same time as the recommendations are learned. This is a form of preprocessing, making it possible to directly multiply the user vector with the course vector in the latent dimension.

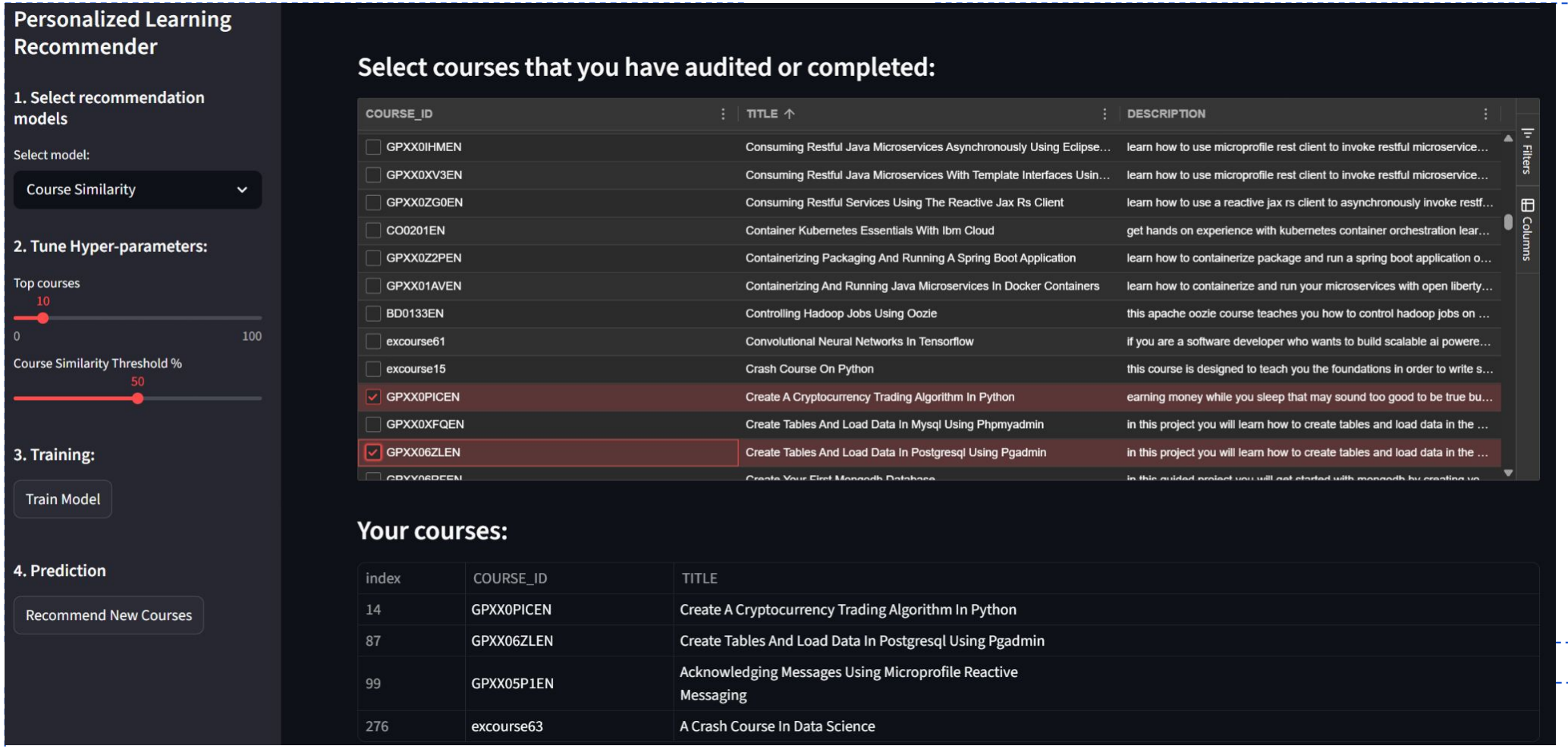


Compare the performance of collaborative-filtering models

- RMSE overview for different collaborative recommender systems.
- The neural network architecture clearly performs best.
- This does come at the cost of higher complexity.
-



Optional: Build a course recommender system app with Streamlit



Conclusions

- Recommender systems can be built in a wide variety of architectures.
- Simple models perform well, and especially with non-critical tasks as recommendations, these are often sufficient.
- For additional performance it is possible to go to more complex architectures such as neural networks.
- Hybrid models using embedding layers can yield improved performance while avoiding the full complexity of a neural network.
- Each architecture provides different balances between number of recommendations adn recommendations choice.

Appendix

- Asset 1
- Asset 2
- Asset 3
- Asset 4
- …

Instructions for learner:

- Include any relevant assets like a GitHub repo, key Python code snippets, charts, notebook outputs, or deployed Streamlit app URLs that you may have created during this project